

Projet de robotique – Rapport final

Année scolaire 2025-2026

"S.T.A.R"

Etudiants : Pelissier Antoine

SOMMAIRE

Introduction	3
Chapitre I : Le fonctionnement	3
I.1. Architecture globale	3
I.1.1 Fonctionnement des tourelles	4
I.2. Schema électrique.....	4
Chapitre II : Le coût totale	5
Chapitre III : Améliorations rapport bibliographique	5
Chapitre IV : Difficultés.....	5
Conclusion et perspectives	6

Introduction

Le projet S.T.A.R. a pour but de sécuriser des zones sensibles potentiellement exposées aux attaques de drones. En effet, ces derniers étant de plus en plus utilisés à des fins militaires ou terroristes, notamment en Ukraine ou en Iran, il devient nécessaire de protéger le ciel contre ces engins peu coûteux, produits en masse et capables de causer de lourds dégâts. La tourelle devra être capable de localiser et de traquer un drone, et de pointer un laser sur lui dans le but éventuel de le neutraliser. Elle devra également pouvoir se déplacer sur tous les terrains, ce système ayant vocation à opérer notamment sur le champ de bataille où le terrain est très souvent accidenté. S.T.A.R. vise donc à proposer une solution mobile, économique et efficace pour contrer cette nouvelle menace technologique.

Chapitre I : Le fonctionnement

I.1. Architecture globale

Le robot fonctionne de la manière suivante (voir Fig. I.1.1 et Fig. I.1.2) : la Jetson Nano et la carte ESP32 se connectent automatiquement au démarrage à un réseau WiFi prédéfini. Les deux appareils étant sur le même réseau, la Jetson peut envoyer des commandes en UDP à l'ESP32. La carte ESP32 s'occupe uniquement des tâches bas niveau : contrôler les quatre servomoteurs ainsi que la vitesse et le sens de rotation des deux moteurs DC via une carte pont en H Cytron MDD10A. La Jetson Nano dispose de deux caméras branchées sur ses ports USB 2.0. Elle exécute un modèle d'IA YOLOv5n optimisé en TensorRT pour détecter les drones en temps réel sur le flux vidéo de chaque caméra. Le flux vidéo annoté avec les détections est ensuite streamer sur une interface web Flask accessible depuis n'importe quel appareil

connecté au réseau. Cette interface intègre également des sliders permettant de contrôler les moteurs de déplacement à distance.

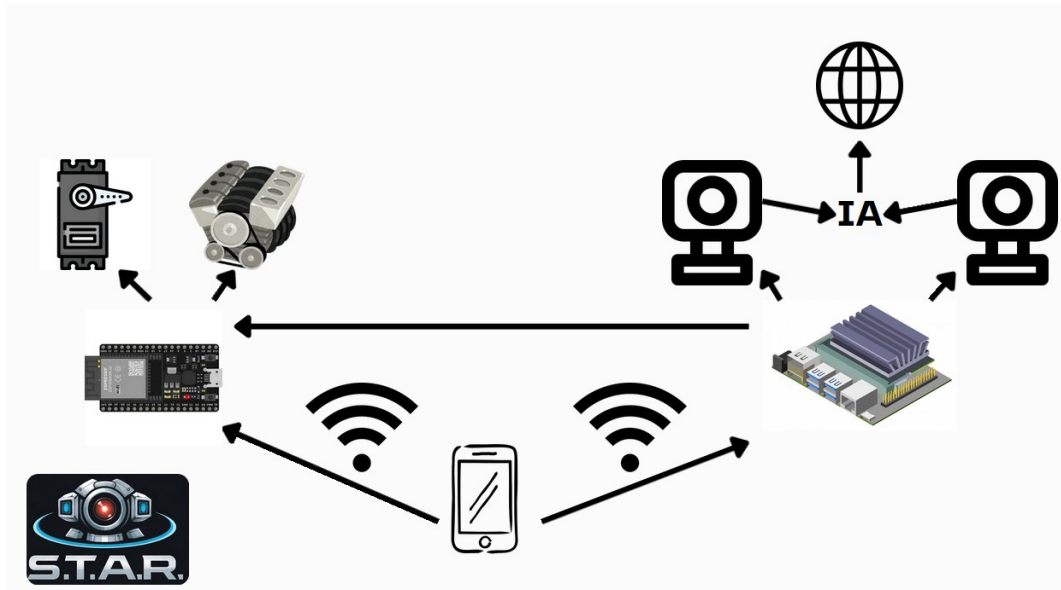


Fig. I.1.1 Fonctionnement schématisé du robot 1

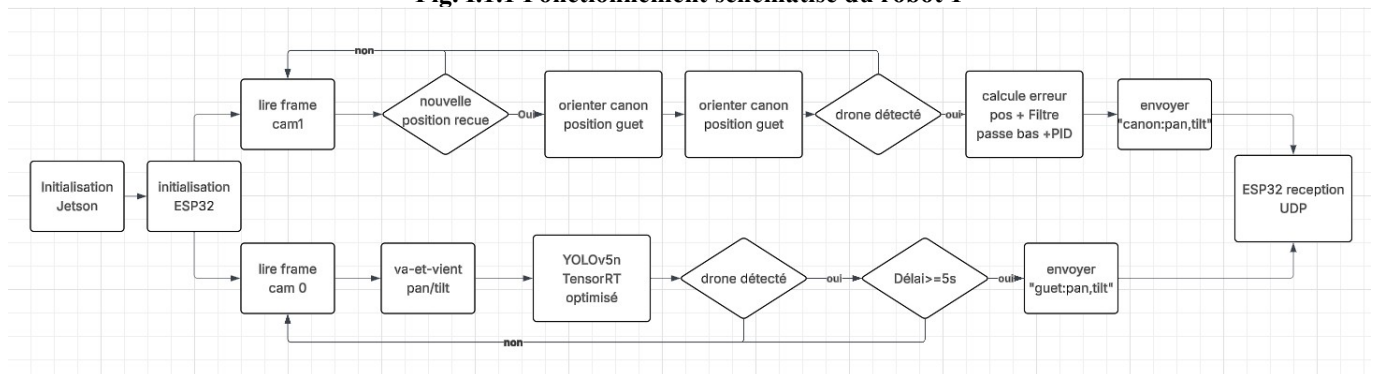


Fig. I.1.2 Algorithme de fonctionnement du robot

I.1.1 Fonctionnement des tourelles

La première tourelle, celle équipée du canon laser, a pour rôle de traquer et de suivre le drone détecté par l'IA. Pour cela, la Jetson Nano calcule en temps réel l'erreur de position entre le centre de la bounding box du drone et le centre de l'image. Un correcteur PID, dont les gains (k_p , k_i , k_d) ont été déterminés de manière empirique, calcule les corrections angulaires à apporter aux servomoteurs pan et tilt. Un filtre passe-bas est appliqué sur l'erreur avant le calcul PID afin de lisser les mouvements et d'éviter les oscillations dues au bruit de détection. Les angles corrigés sont ensuite envoyés à l'ESP32 via UDP au format « canon:pan,tilt ». Le laser s'allume automatiquement dès qu'un drone est détecté et s'éteint sinon.

La seconde tourelle est une tourelle de guet : elle effectue un balayage continu et automatique dans les plans pan (0° à 180°) et tilt (90° à 150°) à la recherche de drones. Dès qu'elle en détecte un, la Jetson mémorise les angles courants des servomoteurs de la tourelle de guet et les transmet toutes les 5 secondes à l'ESP32. La tourelle canon s'oriente alors vers cette position de départ, puis son propre système de détection et de tracking PID prend le relais pour affiner le ciblage.

I.2. Schema électrique

Le schéma électrique représenté en figure I.2.1 représente uniquement la partie électrique de l'ESP32 on voit qu'une alimentation 10V relié a un interrupteur permet l'alimentation des moteurs DC puis on a un BUCK qui permet d'abaisser la tension a 5V afin d'alimenter la carte, les servos mais aussi le laser.

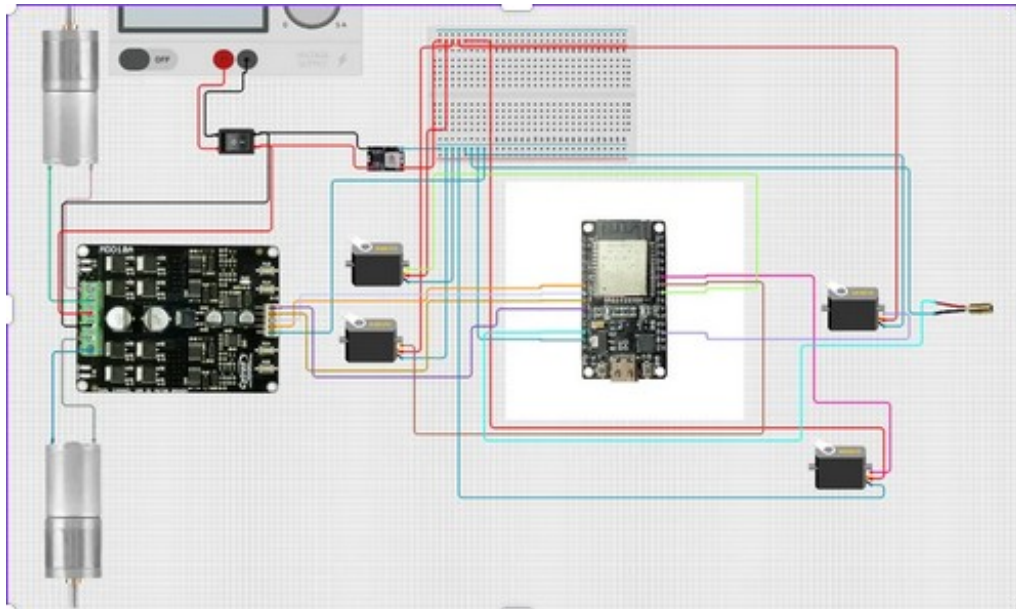


Fig. I.2.1 : Schéma électrique de la partie ESP32 1

Chapitre II : Le coût totale

Composant	Laser	BUCK	Interrupteur	MDD10A	Moteur DC10V	ESP 32	Servos-moteurs	Webcam c505e	Jetson nano
Quantité	1	1	1	1	2	1	4	2	1
Prix unitaire	1	2	1	30	10	5	15	50	250
Totale	469€								

A cela il faut ajouter le temps passé sur le projet avec un cout horaire de 24€ de l'heure nous y avons passer environ 150h en cours mais aussi et surtout en dehors. Cela représente tout de même 3600€ supplémentaires. Nous arrivons donc à un **coût total de 4069€** pour le projet S.T.A.R.

Chapitre III : Améliorations rapport bibliographique

Le rapport bibliographique fait en début d'année nous semble maintenant incomplet en effet aucune recherche sur les IA n'est fait mais aussi du système de tracking en effet c'est une partie non négligeable du projet sur lequel nous n'avons pas assez creusé. Cette partie manquante des recherches nous a conduit à perdre du temps par la suite en cherchant et en partant sur de mauvaise piste quand aux choix des solutions. Le planning prévisionnel lui aussi étant totalement irréaliste en prévoyant que 12h de modélisation 3D ce qui semble maintenant que nous avons plus d'expérience en projet totalement irréalisable.

Chapitre IV : Difficultés

Au cours du projet nous avons rencontré de nombreuses difficultés techniques.

La première et la plus structurante est liée à la version de Python embarquée sur la Jetson Nano. Celle-ci tourne sous Python 3.6.9 via JetPack 4.6, alors que les cartes plus récentes comme la Jetson Orin Nano ou

Orin NX fonctionnent sous Python 3.8 ou 3.10. Cette contrainte a rendu impossible l'utilisation directe d'Ultralytics YOLOv8, qui nécessite Python 3.8 minimum. Il a donc fallu utiliser YOLOv5 v6.2 à la place.

La deuxième difficulté est liée à l'espace disque disponible sur la carte système de la Jetson Nano, qui ne disposait que de 800 Mo libres. Cela a rendu impossible l'installation des bibliothèques Python (PyTorch, TorchVision, YOLOv5) sur le système. La solution a été d'installer l'ensemble des packages sur la carte SD montée en /mnt/sdcard via la variable d'environnement PYTHONUSERBASE, et d'ajouter le chemin correspondant au PYTHONPATH.

La troisième difficulté concerne le format de sortie du modèle. Le modèle YOLOv5n entraîné via Ultralytics récent produit une sortie au format (1, 5, 8400) nommée output0, alors que YOLOv5 v6.2 attendait une sortie nommée output au format (1, 8400, 6). Il a fallu corriger le nom de la sortie dans le code source de YOLOv5 et réécrire entièrement le post-traitement avec un NMS manuel via torchvision.ops.nms, en transposant et reformatant la sortie du modèle.

La quatrième difficulté est le calibrage du correcteur PID. Les sens de rotation des servomoteurs dépendent du montage physique, et plusieurs inversions de signes ont été nécessaires (correction pan inversée, correction tilt inversée) avant d'obtenir un tracking fonctionnel. Les gains ont ensuite été ajustés empiriquement : un kp trop élevé causait des dépassements importants, un kd insuffisant ne permettait pas d'amortir les oscillations.

Enfin, la mise en place du démarrage automatique au boot a nécessité de configurer à la fois la connexion WiFi automatique via NetworkManager (suppression des restrictions user: robot3 sur la connexion) et la connexion automatique de session via GDM3, puis la création d'un service systemd qui lance le script Python après l'établissement du réseau.

Conclusion et perspectives

Nous avons réalisé lors de ce projet un système de double tourelle autonome fonctionnel, capable de détecter et de traquer un drone en temps réel grâce à un modèle YOLOv5n optimisé en TensorRT sur GPU. La tourelle canon assure le suivi par correcteur PID avec filtre passe-bas, le laser s'allume automatiquement à la détection, et l'interface web Flask permet la visualisation et le contrôle à distance. Le démarrage est entièrement automatique au boot sans intervention humaine.

Il reste cependant plusieurs points à améliorer : la base mobile n'est pas encore pleinement opérationnelle, le tracking présente encore quelques saccades dues au faible nombre de FPS de la Jetson Nano, et la détection de la tourelle de guet gagnerait à être affinée. La caméra thermique acquise pour ce projet n'a pas encore été intégrée.

Pour la suite, le remplacement de la Jetson Nano par une Jetson Orin NX permettrait de passer de Python 3.6 à Python 3.10, d'utiliser directement Ultralytics YOLOv8, et d'atteindre un débit d'inférence de 150 à 200 FPS contre 10 à 15 FPS actuellement, rendant le tracking beaucoup plus fluide et réactif. L'utilisation de caméras CSI à 120 FPS à la place des webcams USB actuelles constitue également une amélioration matérielle significative.

BIBLIOGRAPHIE

[1] YOLOv5n <https://docs.ultralytics.com/models/yolov5>

[2] NVIDIA Jetson software <https://developer.nvidia.com/embedded/develop/software>

[3] FLASK documentation <https://flask.palletsprojects.com/en/stable/>